

TfidfVectorizer#

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Description:

Convert a collection of raw documents to a matrix of TF-IDF features. Equivalent to `CountVectorizer` followed by `TfidfTransformer`. For an example of usage, see [Classification of text documents using sparse features](#). For an efficiency comparison of the different feature extractors, see [FeatureHasher and DictVectorizer Comparison](#).

Function Signature

```
class sklearn.feature_extraction.text.TfidfVectorizer(*,
input='content', encoding='utf-8', decode_error='strict',
strip_accents=None, lowercase=True, preprocessor=None,
tokenizer=None, analyzer='word', stop_words=None,
token_pattern='(?u)\\b\\w\\w+\\b', ngram_range=(1, 1),
max_df=1.0, min_df=1, max_features=None, vocabulary=None,
binary=False, dtype=<class 'numpy.float64'>, norm='l2',
use_idf=True, smooth_idf=True, sublinear_tf=False)[source]#
```

Parameters:

`input`{'filename', 'file', 'content'}, default='content'

```
encodingstr, default='utf-8'
```

```
decode_error{'strict', 'ignore', 'replace'},  
default='strict'
```

```
class sklearn.feature_extraction.text.TfidfVectorizer(*,  
input='content', encoding='utf-8', decode_error='strict',  
strip_accents=None, lowercase=True, preprocessor=None,  
tokenizer=None, analyzer='word', stop_words=None,  
token_pattern='(?u)\\b\\w+\\b', ngram_range=(1, 1),  
max_df=1.0, min_df=1, max_features=None, vocabulary=None,  
binary=False, dtype=<class 'numpy.float64'>, norm='l2',  
use_idf=True, smooth_idf=True, sublinear_tf=False)[source]#
```

Parameters:

```
input{'filename', 'file', 'content'}, default='content'
```

```
encodingstr, default='utf-8'
```

```
decode_error{'strict', 'ignore', 'replace'},  
default='strict'
```

```
build_analyzer()[source]#
```

Returns:

```
analyzer: callable
```

```
build_preprocessor()[source]#
```

Returns:

Parameters

`encodingstr, default='utf`

If bytes or files are given to analyze, this encoding is used to decode.

`input_featuresarray`

Not used, present here for API consistency by convention.

`paramsdict`

Parameter names mapped to their values.

`X{array`

Document-term matrix.

`**paramsdict`

Estimator parameters.

Code Examples

```
>>> from sklearn.feature_extraction.text import TfidfVectorizer
>>> corpus = [
...     'This is the first document.',
...     'This document is the second document.',
...     'And this is the third one.',
...     'Is this the first document?',
... ]
>>> vectorizer = TfidfVectorizer()
>>> X = vectorizer.fit_transform(corpus)
>>> vectorizer.get_feature_names_out()
array(['and', 'document', 'first', 'is', 'one', 'second', 'the',
      'third',
      'this'], ...)
>>> print(X.shape)
(4, 9)
```

Notes & Warnings

NOTE See also `CountVectorizer` Transforms text into a sparse matrix of n-gram counts. `TfidfTransformer` Performs the TF-IDF transformation from a provided matrix of counts.

Detailed Documentation

Documentation

Convert a collection of raw documents to a matrix of TF-IDF features.

Equivalent to `CountVectorizer` followed by `TfidfTransformer`.

For an example of usage, see [Classification of text documents using sparse features](#).

For an efficiency comparison of the different feature extractors, see [FeatureHasher and DictVectorizer Comparison](#).

For an example of document clustering and comparison with `HashingVectorizer`, see [Clustering text documents using k-means](#).

Read more in the [User Guide](#).

If 'filename', the sequence passed as an argument to `fit` is expected to be a list of filenames that need reading to fetch the raw content to analyze.

If 'file', the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory.

If 'content', the input is expected to be a sequence of items that can be of type string or byte.

If bytes or files are given to analyze, this encoding is used to decode.

Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given encoding. By default, it is 'strict', meaning that a `UnicodeDecodeError` will be raised. Other values are 'ignore' and 'replace'.

Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII

mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed.

Both 'ascii' and 'unicode' use NFKD normalization from `unicodedata.normalize`.

Convert all characters to lowercase before tokenizing.

Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if analyzer is not callable.

Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if analyzer == 'word'.

Whether the feature should be made of word or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space.

If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input.

Changed in version 0.21: Since v0.21, if input is 'filename' or 'file', the data is first read from the file and then passed to the given callable analyzer.

If a string, it is passed to `_check_stop_list` and the appropriate stop list is returned. 'english' is currently the only supported string value. There are several known issues with 'english' and you should consider an alternative (see Using stop words).

If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if analyzer == 'word'.

If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra

corpus document frequency of terms.

Regular expression denoting what constitutes a “token”, only used if `analyzer == 'word'`. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator).

If there is a capturing group in `token_pattern` then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.

The lower and upper boundary of the range of `n`-values for different `n`-grams to be extracted. All values of `n` such that `min_n <= n <= max_n` will be used. For example an `ngram_range` of (1, 1) means only unigrams, (1, 2) means unigrams and bigrams, and (2, 2) means only bigrams. Only applies if `analyzer` is not callable.

When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not `None`.

When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not `None`.

If not `None`, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used.

This parameter is ignored if vocabulary is not `None`.

Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents.

If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set binary to True, use_idf to False and norm to None to get 0/1 outputs).

Type of the matrix returned by fit_transform() or transform().

Each output row will have unit norm, either:

'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied.

'l1': Sum of absolute values of vector elements is 1. See normalize.

None: No normalization.

Enable inverse-document-frequency reweighting. If False, $\text{idf}(t) = 1$.

Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.

Apply sublinear tf scaling, i.e. replace tf with $1 + \log(\text{tf})$.

A mapping of terms to feature indices.

True if a fixed vocabulary of term to indices mapping is provided by the user.

Inverse document frequency vector, only defined if use_idf=True.

Transforms text into a sparse matrix of n-gram counts.

Performs the TF-IDF transformation from a provided matrix of counts.

Return a callable to process input data.

The callable handles preprocessing, tokenization, and n-grams generation.

A function to handle preprocessing, tokenization and n-grams generation.

Return a function to preprocess the text before tokenization.

A function to preprocess the text before tokenization.

Return a function that splits a string into a sequence of tokens.

A function to split a string into a sequence of tokens.

Decode the input into a string of unicode symbols.

The decoding strategy depends on the vectorizer parameters.

The string to decode.

A string of unicode symbols.

Learn vocabulary and idf from training set.

An iterable which generates either str, unicode or file objects.

This parameter is not needed to compute tfidf.

Learn vocabulary and idf, return document-term matrix.

This is equivalent to fit followed by transform, but more efficiently implemented.

An iterable which generates either str, unicode or file objects.

This parameter is ignored.

Tf-idf-weighted document-term matrix.

Get output feature names for transformation.

Not used, present here for API consistency by convention.

Transformed feature names.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Build or fetch the effective stop words list.

A list of stop words.

Return terms per document with nonzero entries in X.

Document-term matrix.

List of arrays of terms.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline).

The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Estimator parameters.

Transform documents to document-term matrix.

Uses the vocabulary and document frequencies (df) learned by `fit` (or `fit_transform`).

An iterable which generates either str, unicode or file objects.

Tf-idf-weighted document-term matrix.

Topic extraction with Non-negative Matrix Factorization and Latent Dirichlet Allocation

Biclustering documents with the Spectral Co-clustering algorithm

Column Transformer with Heterogeneous Data Sources

Sample pipeline for text feature extraction and evaluation

Classification of text documents using sparse features

Clustering text documents using k-means

FeatureHasher and DictVectorizer Comparison

`sklearn.feature_selection`

© Copyright 2007 - 2025, scikit-learn developers (BSD License).